COMPOSITIONAL CERTIFICATES FOR REVERSIBLE NEURAL NETWORKS

BY ANONYMOUS AUTHOR

We give a checked certificate calculus for finite reversible networks over fixed-width tensors. A coupling layer preserves one tensor and adds an arbitrary function of it into the other. Its inverse replaces addition by subtraction. The update function is opaque to the reversibility argument. We prove local cancellation, composition, and a linear-size certificate bound. The Roop Lean development checks the generic theorems for every tensor width, element width, layer direction, and finite layer list. The result removes expanded tensor arithmetic from the network round-trip proof. It does not yet prove that generated Weave code implements the certificate representation.

1. Introduction. Coupling transformations make an inverse available without inverting the function used in the update. This idea appears in NICE [2] and reversible residual networks [3]. Roop uses the same algebra over machine integers and generates inverse programs. The verification problem differs from a numerical reversibility argument. It must establish exact recovery of bit patterns and avoid expanding a large generated tensor computation merely to prove cancellation.

This paper gives a small certificate language for that purpose. Its central theorem is generic in network depth. The proof term records the composition of layer certificates, not the internal arithmetic of the update functions. Lean checks the implementation [1].

2. Certified transformations.

DEFINITION 1. For types A and B, a reversible transformation $r \in \mathrm{Rev}(A, B)$ consists of functions

$$r.\mathrm{run} : A \to B, \qquad r.\mathrm{undo} : B \to A$$

such that, for every $a \in A$ and $b \in B$,

$$r.\mathrm{undo}(r.\mathrm{run}(a)) = a, \qquad r.\mathrm{run}(r.\mathrm{undo}(b)) = b.$$

Let $B_n = \mathrm{BitVec}(n)$ and $T_{d,n} = \mathrm{Fin}(d) \to B_n$. Addition and subtraction on B_n are modulo 2^n. Thus overflow is part of the model rather than an exceptional case.

DEFINITION 2. For $f : T_{d,n} \to T_{d,n}$, define two coupling updates:

$$U_f(q, p) = (q, p + f(q)), \qquad V_f(q, p) = (q + f(p), p),$$

where addition is pointwise. Their proposed inverse maps replace $+$ by $-$.

LEMMA 3 (local cancellation). *For every d, n, and $f : T_{d,n} \to T_{d,n}$, both U_f and V_f are elements of* $\mathrm{Rev}(T_{d,n} \times T_{d,n}, T_{d,n} \times T_{d,n})$.

MSC2020 subject classifications. Primary 68N20.
Keywords and phrases. reversible computation, formal verification, neural networks.

PROOF. Consider U_f. Its inverse receives $(q, p + f(q))$, preserves q, and returns

$$(p + f(q)) - f(q) = p$$

at each tensor index. The other round trip uses $(p - f(q)) + f(q) = p$. Both equations hold in B_n. Function extensionality lifts the scalar equations to tensors. The proof for V_f swaps the two components. □

The lemma makes no assumption about f. It may compute a matrix product, activation, convolution, attention block, or another network. The preserved component makes f available again during reversal.

3. Networks as programs. A program is either empty or a certified transformation followed by a program. Define execution by applying transformations from head to tail and reverse execution by applying their inverses from tail to head.

THEOREM 4 (compositional round trip). *For every finite program P of transformations in* $\mathrm{Rev}(A, A)$ *and every $s \in A$,*

$$\mathrm{undo}_P(\mathrm{exec}_P(s)) = s, \qquad \mathrm{exec}_P(\mathrm{undo}_P(s)) = s.$$

PROOF. Induct on P. The empty program has identity execution. For $P = r :: Q$, reverse execution first applies the induction hypothesis to Q, then applies $r.\,\mathrm{undo}$ to $r.\,\mathrm{run}(s)$. Local cancellation gives the first equality. The other equality is symmetric. □

COROLLARY 5 (arbitrary finite coupling networks). *For every finite list of functions and directions, the network obtained by choosing U_f or V_f at each position has both round trips.*

PROOF. Apply Lemma 3 to each list element and then Theorem 4. □

THEOREM 6 (certificate representation bound). *Suppose each layer certificate is supplied by name. A certificate for a network of k layers contains $O(k)$ composition nodes and no subterm representing the body of an update function.*

PROOF. The program constructor adds one node and one reference to a supplied layer certificate per layer. The proof of Theorem 4 follows the same list structure. It invokes, but does not unfold, the two equations stored in each certificate. □

This is a statement about the certificate representation. It does not claim a bound on every Lean elaboration strategy or on the cost of validating a concrete update contract.

4. Checked development and boundary. The Lean module `TensorNetwork.lean` defines the layer representation and both round-trip theorems. A Rust regression invokes Lean on the module. These checks establish the generic certificate calculus.

The `roop weave -certificate` command emits a Lean file containing the transformation directions for a model and one opaque update symbol for each transformation. Lean checks its round-trip theorems. The certificate and the emitted Roop source come from the same model description, but this common origin is not a refinement proof. The missing correspondence theorem must show that a generated layer leaves its stated source tensor unchanged and performs the modular pointwise update named by the certificate. In particular, it must connect each opaque update symbol to generated Einsum code, temporary cleanup, and alias conditions. Until then, the development proves the calculus and checks a model plan, not a refinement theorem for the emitted compiler output.

REFERENCES

[1] L. de Moura and S. Ullrich (2021). The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, 625–635.

[2] L. Dinh, D. Krueger and Y. Bengio (2015). NICE: Non-linear independent components estimation. *International Conference on Learning Representations*.

[3] A. N. Gomez, M. Ren, R. Urtasun and R. B. Grosse (2017). The reversible residual network: Backpropagation without storing activations. In *Advances in Neural Information Processing Systems 30*.